TRON Programming Contest 2026

Program Plan Submission

RTOS Application Category | Student Division

Submission Date:	March 31, 2026	**Board Preference:**	STM32N6570-DK / EK-RA8P1
Category:	RTOS Application (Student)	**Team Size:**	4 Members

1. Application Program Name

MedSight

2. Program Overview — Purpose and Functions

Background & Problem Statement

Medication non-adherence is a pervasive and costly global healthcare challenge. Patients, particularly the elderly, those managing multiple chronic conditions, or individuals living alone, frequently miss doses, take incorrect medications, or consume wrong quantities. This leads to worsened health outcomes, preventable hospital readmissions, and significant burden on caregivers and healthcare systems. Existing reminder solutions rely on manual input or simple timers that lack the ability to verify whether the correct medicine was actually taken.

What MedSight Does

MedSight is a camera-based, AI-powered medication adherence assistant that runs fully on-device, powered by the μT-Kernel 3.0 RTOS. Using the board's integrated camera, MedSight captures an image of the medication presented by the user and performs real-time visual classification to identify the pill, capsule, or medicine packet. It then cross-references the identified item against a pre-configured schedule to verify that the correct medication and dosage are being taken at the right time.

Core Functions

- Real-time camera capture triggered by a scheduled RTOS task or user button press.
- On-device CNN-based pill/packet visual classification using the board's NPU.
- Schedule validation - compares identified medicine against the patient's stored plan.
- Audio and/or visual alert feedback on incorrect medication, missed dose, or confirmation.
- Caregiver notification flag stored in non-volatile memory for later review.
- Fully offline operation - no cloud connectivity required, preserving patient privacy.

3. Development Plan

The project will be developed by a team of **4 members**. Responsibilities are divided as follows, though collaboration across areas is expected:

Role	Responsibilities
Member 1: Lead / RTOS Integration	µT-Kernel 3.0 task architecture, RTOS scheduler configuration, system integration, and overall project coordination.
Member 2: AI / Vision	CNN model selection, quantization, on-device inference pipeline, and NPU integration.
Member 3: Firmware / Drivers	Camera driver, GPIO/interrupt handling, peripheral BSP, and non-volatile storage.
Member 4: UI / Testing	Audio-visual feedback output, patient schedule configuration interface, and system-level testing/validation.

4. Development Environment and Programming Language

Item	Details
Host OS	Windows 11
IDE	STM32CubeIDE / e² studio / VS Code with Cortex-Debug
Toolchain	GNU Arm Embedded Toolchain
RTOS	µT-Kernel 3.0
AI Framework	STM32Cube.AI / Renesas RA Smart Configurator + TensorFlow Lite for Microcontrollers (TFLM)
Programming Languages	C and Python
Version Control	Git / GitHub
Debugging	JTAG/SWD via on-board ST-LINK / J-Link

5. Development Scope (Estimated)

Module	Description	Est. Lines
RTOS Task Architecture	Task definitions, scheduling, semaphores, message queues	~800
Camera Driver (BSP)	Peripheral init, frame capture, DMA pipeline	~600
AI Inference Engine	TFLM integration, NPU offload, pre/post-processing	~700
Schedule Manager	Patient config, time-match logic, NVM read/write	~500
Alert / Feedback Module	Buzzer, LED, optional small display output	~300
Utilities / HAL Glue	Board support, logging, helper functions	~400
Total (estimated)		**~3,300**

6. Feature Description

System Architecture Overview

MedSight is structured around four concurrent µT-Kernel 3.0 tasks communicating via message queues and semaphores. The design ensures hard real-time guarantees for time-critical operations (frame capture, inference trigger) while lower-priority tasks handle storage and UI.

[Camera Task]	→	[Inference Task]	→	[Validation Task]	→	[Alert Task]
Captures frame on button/schedule trigger		Runs CNN model via NPU		Checks against patient schedule		Buzzer / LED / Display output

Detailed Feature Breakdown

Scheduled & On-Demand Capture: The Camera Task wakes either on a scheduled µT-Kernel alarm (aligned to dose times) or on user button press. A frame is captured via the camera interface and placed on a shared message queue.

On-Device AI Inference: A lightweight CNN model (MobileNetV2 or custom architecture), quantized to INT8, is deployed using STM32Cube.AI or TFLM. The Inference Task consumes camera frames and outputs a classification label (pill/packet type) with a confidence score, offloading computation to the onboard NPU.

Schedule Validation: The Validation Task compares the inference result against the patient's medication schedule stored in NVM. It determines pass/fail (correct medicine, correct time window) and passes a structured result to the Alert Task via message queue.

Audio-Visual Feedback: The Alert Task drives a buzzer and LED (green = correct, red = incorrect/missed) and optionally pushes a text message to a small display. Missed or incorrect events are logged to NVM with a timestamp for caregiver review.

Privacy-First, Offline Design: All computation, model inference, and data storage occur entirely on-device. No network stack, no cloud upload. Patient medication data never leaves the embedded system.

7. Highlights of the Program

TRON × AI Synergy: MedSight exemplifies the contest's theme by tightly coupling µT-Kernel 3.0's deterministic real-time scheduling with on-device AI inference, not as loosely connected components, but as a unified, task-orchestrated system where the RTOS is responsible for the precise timing of each AI pipeline stage.

Hardware-Accelerated Inference on Bare-Metal RTOS: Unlike existing medication apps that rely on smartphone cameras and cloud APIs, MedSight demonstrates that meaningful AI-powered healthcare tooling can run on a constrained embedded platform with deterministic latency, a genuinely novel application of RTOS + edge AI.

No Cloud, No Compromise: The privacy-first architecture removes the most common barrier to adoption of AI health tools in sensitive home environments. Everything runs offline, making MedSight viable even without internet access.

Open Source Commitment: The full source code, model training scripts, and documentation are planned to be released openly upon submission, encouraging community reuse and further development.

Extensibility: The modular RTOS task design makes MedSight straightforward to extend future work could include multi-patient support, additional sensor fusion (weight sensors for blister pack detection), or Bluetooth-based caregiver notifications.

8. Applicant's Strengths

As an undergraduate student with a deep passion for embedded systems and hardware design, I bring a combination of technical breadth, hands-on project experience, and demonstrated leadership to this contest.

Prior TRON Contest Experience

I participated in the TRON Programming Contest 2025, where I gained practical, first-hand experience developing on μT-Kernel 3.0 — from understanding the kernel's task model and synchronization primitives to debugging real-time scheduling issues on physical hardware. That experience gave me a strong foundation in how to structure RTOS applications effectively, and MedSight builds directly on those learnings with increased ambition and technical depth.

Technical Expertise

- **Embedded C & Firmware:** Comfortable writing bare-metal and RTOS-based firmware, developing peripheral drivers, and managing memory in constrained environments.
- **RTOS / μT-Kernel:** Hands-on experience with task creation, semaphores, message queues, timers, and interrupt handling under μT-Kernel 3.0.
- **VLSI & Digital Design:** Background in digital circuit design, hardware description languages, and understanding of processor-level architecture, informing how I approach hardware-software co-design.
- **AI on Edge Devices:** Experience deploying and optimizing quantized ML models on microcontroller platforms, including understanding NPU offloading and inference pipeline design.

Leadership & Collaboration

I have led multiple hardware and embedded software projects in academic settings, coordinating cross-functional teams, managing timelines, and integrating contributions across firmware, hardware, and software layers. I am comfortable both driving technical decisions and supporting teammates in overcoming implementation challenges.

Motivation

I am driven by the intersection of real-time embedded systems and AI, the idea that powerful, intelligent behavior can be delivered reliably by small, constrained hardware without any cloud dependency. The TRON contest, with its unique focus on μT-Kernel 3.0 and real-world application, is precisely the right arena to push that ambition further. MedSight is a project I genuinely care about, it addresses a real, widespread problem using technology I am passionate about. I look forward to the challenge.